

Enmanuel Morillo

9/12/2026

CSC-271

1. Which type of ideas do you think apply to Shyp? Why?

I believe the main idea types that apply to Shyp are Simplify, Virtualize, and Remix. Shyp was created to make the shipping process easier and more convenient for users. Instead of having to find a box, package an item, choose a shipping company, and travel to a shipping location, users could request someone to pick up the item and handle most of the shipping process for them.

Shyp fits the Simplify category because it takes a process that normally requires several steps and makes it much easier for the user. It also fits Virtualize because parts of the process, such as requesting a pickup and tracking the package, can be handled through an app. Lastly, it fits Remix because it combines traditional shipping services with the convenience of an on-demand service.

2. Which type of ideas do you think apply to Curofy? Why?

I think Simplify, Me Too, Virtualize, and Remix all apply to Curofy. Curofy is a professional social networking platform designed specifically for doctors. It gives medical professionals a place to communicate, discuss cases, ask for second opinions, and connect with other doctors.

Curofy fits Simplify because it makes it easier for doctors to communicate and share information with each other. It also fits Me Too because it takes an idea similar to platforms such as LinkedIn and applies it to a more specific market, which in this case is the medical field.

It fits Virtualize because doctors can communicate and discuss cases online instead of depending only on traditional forms of communication. Lastly, it fits Remix because it combines social networking, professional networking, and medical collaboration into one platform.

3. Product Hunt Startup: Harden

Describe the idea.

The startup I chose from Product Hunt is Harden. Harden is a security tool designed for AI coding agents. Its main purpose is to check what an AI coding agent is attempting to do before the action is actually performed.

For example, if an AI agent tries to delete an important file, access private information, or perform an action that the user did not request, Harden can detect the behavior and stop it. The program can also run locally, which helps users keep their code and other information more private.

Describe the type(s) of ideas that apply to it.

I think Harden mainly fits the Simplify and Remix categories.

It fits Simplify because developers do not have to manually review every action that an AI coding agent is attempting to perform. Harden helps automate that process while adding another layer of security.

It also fits Remix because it combines two areas that already exist: cybersecurity and AI coding tools. Instead of creating a normal security program or another AI coding assistant, Harden combines the two ideas to solve a newer problem created by AI agents having greater access to a user's computer.

Is the idea a painkiller or a vitamin? Does it address users' needs or emotions? Why?

I would consider Harden a painkiller because it solves a real problem developers may experience. AI coding agents can have access to important files, terminals, passwords, and other sensitive information. If an AI agent makes a mistake or performs an action the user did not intend, it could create a serious security problem.

Harden helps prevent this by checking actions before they are performed. Because of this, I think it addresses an actual need rather than only providing an optional feature.

Is the idea easy to monetize?

I think Harden would be fairly easy to monetize. Individual developers could use a basic version for free, while businesses and larger development teams could pay for more advanced features.

For example, companies could pay for better monitoring, security reports, team management features, and additional integrations. Since companies already spend money on cybersecurity, I think many businesses would be willing to pay for a product that helps protect their code, files, and systems from unwanted AI actions.

Is the idea simple and easy to describe?

Yes, I think Harden is fairly simple to explain. Harden is basically a security tool that checks what AI coding agents are trying to do before they are allowed to perform those actions.

Even though the technology behind the product may be complicated, the main purpose is easy for users to understand.

Is the idea personally relevant?

Yes, Harden is personally relevant to me because I am a Computer Science major and regularly work with programming and software development. AI tools are also becoming more common in programming, so I think developers will have to become more aware of how much access they give these tools.

A product like Harden could be useful to me in the future because it would allow me to use AI coding tools while still having another layer of protection between the AI and my computer.

Does the idea have a large market?

I think Harden has the potential to have a large market. AI coding tools are becoming more common among individual developers, startups, and larger companies. As these tools become more advanced, developers will probably give them more access and responsibility.

This creates a greater need for tools that can monitor what AI agents are doing. Harden could potentially be useful to individual programmers, software companies, startups, and larger businesses that rely on AI-assisted development.

Does the idea have a legitimate secret sauce?

I think Harden does have a legitimate secret sauce. One of the main things that makes it different is that it checks the actions of an AI coding agent before they actually happen instead of only scanning the code afterward.

It also considers what the user originally asked the AI to do and the context of the current task. Another advantage is that Harden can run locally, meaning private code and information can remain on the user's computer.

I think these features help separate Harden from more traditional cybersecurity or code-scanning tools.

4. Brainstorming

Problems I considered

Before choosing my final product idea, I thought about several different problems that could potentially be solved through a website or software application.

One problem I considered was students having difficulty finding other students to study with. Students may know people in their classes, but their schedules may not match, or they may not know who is actually interested in studying together.

Another problem I thought about was losing items around a college campus. Items such as AirPods, keys, wallets, chargers, and student IDs can easily be lost, and there is not always a simple way to connect the person who lost an item with the person who found it.

I also considered problems with group projects. In many group projects, one or two people may end up doing most of the work while other members contribute much less. It can also be difficult to keep track of who completed each part of the project.

The problem that interested me the most was software bug reporting. People experience bugs and glitches all the time while using websites, applications, games, and other software. However, many regular users do not know how to clearly explain what happened.

For example, a user might report a problem by saying, "the app doesn't work" or "the page is broken." While the user knows something went wrong, that description usually does not give a developer enough information to understand or recreate the issue.

Possible Solutions

For the study problem, one solution I considered was a website that matches students based on their classes, schedules, and preferred ways of studying.

For the lost item problem, I thought about creating a campus lost-and-found website where students could report lost and found items. The website could compare information such as item type, location, date, color, and description to help users find possible matches.

For the group project problem, I considered creating a website where each member records the work they completed. The website could create a timeline showing each person's contribution throughout the project.

For the software bug reporting problem, I thought about creating a website that guides users through the process of reporting a bug. Instead of expecting the user to understand technical terminology, the website would ask simple questions about what happened.

It could ask what device the person was using, what application or website they were using, what they were trying to do, what steps they took before the problem happened, what they expected to happen, and what actually happened.

After the user answers the questions, the website could turn their responses into an organized bug report that would be easier for a developer to understand.

Which idea did I choose and why?

Out of all of the ideas I considered, I decided to choose the software bug reporting idea. My product will be called BugFlow.

I chose BugFlow because it relates the most to me personally. As a Computer Science student, I regularly work with software, websites, and programming. I also tend to notice bugs and glitches in software, sometimes without intentionally looking for them.

Because I have experience with programming, I understand that developers need detailed information in order to recreate and fix a bug. I thought it would be interesting to create a product that helps regular users provide developers with that information in a much easier way.

I also chose BugFlow because it gives me the opportunity to create an interactive website instead of a site that only displays information.

Is the idea new or has it existed before?

The general idea of bug reporting already exists. Software companies already use bug-tracking systems, support forms, and other tools to collect information about software problems.

However, BugFlow would take a slightly different approach. Instead of designing the system mainly for developers or other technical users, BugFlow would be designed for everyday users who may have little or no understanding of software development.

The goal would be to make creating a useful bug report as simple as answering a series of normal questions.

What industry would the idea be useful in? Why?

BugFlow would mainly be useful in the technology and software development industry.

Software companies need reliable ways for users to report problems with their products. BugFlow could also be useful in quality assurance because QA testers spend a large amount of time finding, reproducing, and documenting software problems.

Customer support teams could also benefit from BugFlow because they often receive complaints from users who know they are experiencing a problem but may not know how to clearly describe it.

5. Product Proposal

Describe the product and the type(s) of ideas that apply to it.

My product is called BugFlow. BugFlow would be a web-based application that helps regular users create detailed and organized software bug reports without needing a technical background.

Instead of opening a traditional form and expecting the user to explain everything on their own, BugFlow would guide the user through the process one step at a time.

For example, the website could ask:

- What website or application were you using?
- What device were you using?
- What operating system were you using?
- What were you trying to do?
- What steps did you take before the problem happened?
- What did you expect to happen?
- What actually happened?
- Did you receive an error message?
- Can you upload a screenshot of the problem?

After the user answers the questions, BugFlow would organize the responses into a complete bug report that could be copied, downloaded, or sent to a developer.

I think the main idea types that apply to BugFlow are Simplify and Remix.

BugFlow fits Simplify because it takes a process that may seem confusing or technical and breaks it down into a simple step-by-step process.

It also fits Remix because it combines ideas from bug-tracking systems, customer support forms, and guided questionnaires into one product designed for regular users.

How is your product personally relevant to you?

BugFlow is personally relevant to me because I am a Computer Science major and regularly work with programming, websites, and different types of software.

I also tend to notice bugs and glitches while using software. Sometimes I will find something that is not working correctly without even looking for a problem.

Because I study Computer Science, I understand that there is a big difference between simply saying "something is broken" and actually explaining what happened.

A developer needs information about what the user was doing, what happened before the problem, what the user expected to happen, and what actually occurred.

I think BugFlow would be useful because it would help regular users provide this information even if they have no programming experience.

What problem will your product solve?

BugFlow will solve the problem of unclear and incomplete software bug reports.

Regular users may know that something went wrong, but they may not know what information is important when reporting the problem.

If a developer does not receive enough information, they may not be able to recreate the bug. If they cannot recreate it, figuring out what caused the problem and how to fix it becomes much more difficult.

BugFlow would help solve this problem by guiding the user through the important details and organizing everything into a structured report.

This would make communication between users and developers more efficient.

What type of website will your product be?

BugFlow would mainly be a web application.

The website would have a homepage that explains what BugFlow does and why detailed bug reports are important.

The main feature of the site would be an interactive Report a Bug section. Users would move through several steps and answer one or more questions on each screen.

At the end of the process, JavaScript could take the user's answers and automatically generate a completed bug report.

The website could also include an About page, Features page, and example bug reports to help users understand how the product works.

Is your product a painkiller or a vitamin?

I would consider BugFlow a painkiller because it solves a real problem rather than only providing an optional feature.

Software companies need accurate information when something goes wrong with their product, while users need a simple way to explain the problem they experienced.

BugFlow helps both sides. It gives the user a simpler reporting process while giving the developer more useful information about the issue.

Is your product easy to monetize?

I think BugFlow could be fairly easy to monetize if it became a real product.

A basic version could remain free for regular users, allowing individuals to create simple bug reports without paying.

Businesses and development teams could pay for a professional version with additional features. These features could include custom bug report forms, company branding, saved reports, dashboards, analytics, team accounts, and integrations with software development tools.

Businesses could also pay to integrate BugFlow directly into their own websites or applications, allowing their customers to report problems without leaving the product.

Is your product simple and easy to describe?

Yes, I think BugFlow is simple and easy to describe.

BugFlow is a website that helps regular users turn software problems into organized bug reports that developers can understand.

I think someone could understand the main purpose of the product from that sentence without needing a technical explanation.

The name BugFlow also fits the idea because the website guides the user through the entire flow of reporting a software bug.

Who will be the likely users of your product? Does your product have a large market?

The main users of BugFlow would be people who experience software problems but may not know how to properly report them.

Other potential users could include software developers, QA testers, customer support teams, startups, technology companies, and businesses that create websites or applications.

I think BugFlow has the potential to serve a large market because software is used almost everywhere.

People use mobile applications, websites, computer programs, video games, banking apps, school websites, streaming platforms, and many other types of software every day. All of these products can experience bugs.

As long as people continue using software, companies will continue needing effective ways for users to report problems.

Does your product have a legitimate secret sauce?

I think BugFlow does have a legitimate secret sauce.

The main feature that would make BugFlow different is that it would be designed around the person experiencing the bug instead of assuming that the person already understands software development.

A normal bug-reporting system might ask for things such as reproduction steps, expected behavior, actual behavior, system environment, or other technical information. Someone without programming experience may not understand what those terms mean.

BugFlow would collect the same information but ask the questions using normal language.

For example, instead of asking for "steps to reproduce," BugFlow could ask:

"Walk us through exactly what you did before the problem happened."

Instead of asking for "expected behavior," it could ask:

"What were you expecting to happen?"

Another useful feature could be a visual timeline that allows the user to list each step they took before the bug occurred.

BugFlow could also review the completed report before it is submitted and warn the user if important information appears to be missing.

I think BugFlow's main secret sauce would be taking a technical process and making it simple enough for almost anyone to understand and use.

